AOSP 进阶丛书 · 第十六册 · ANDROID 13 (API 33)

车机 AAOS 源码精读

CarService 微服务容器（ProxiedService:40）、VHAL 车信号属性模型（车速→禁止视频全链）、车机电源状态机（熄火≠关机）、多显示与乘客屏策略。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目录

前言：从手机到四个轮子 2
第一章 CarService：车机的微服务容器 2
第二章 VHAL：车信号的统一语言 3
第三章 车机电源状态机 4
第四章 多显示与乘客屏 4
第五章 设计决策六问 5
附录 A 观测与实战 5
附录 B 快问快答（12 条） 5
附录 C 自检题 6
第六章 启动链：CarServiceImpl 到 ICarImpl.init 的微服务装配 6
第七章 VHAL 属性模型：prop 位结构、areaId 与分发表 10
第八章 属性订阅链：registerListener 到 onPropertyEvent 的完整闭环 13
第九章 CarOccupantZoneService：座位-屏幕-用户的三角绑定 18
教科书习题集（分级） 22
术语表 25
排障手册：车机 AAOS问题定位 27
车机 AAOS 核心速查表 29

本册速览

CarServiceImpl:40 extends ProxiedService（车机微服务容器）
VHAL（VehicleHal.java）：车规级 HAL——车速/档位/空调等全部车信号
CarPowerManagementService：车机电源状态机（与手机 PowerManager 差异）
多显示与乘客屏：每屏独立 ActivityDisplay
设计决策：为什么车机不叫 Android Auto

前言：从手机到四个轮子

AAOS（Android Automotive OS）不是"手机版 Android 装上车"——它是**同一内核/框架+完全不同的产品层**：车规信号（车速/档位/方向盘）经 VHAL 进来、电源语义完全不同（车熄火≠关机）、多屏是标配（仪表+中控+乘客屏）。本册讲清 CarService 容器、VHAL 车信号层、车机电源状态机与多显示。

源码基线不变。主角：`packages/services/Car/`（整个车机服务）。

第一章 CarService：车机的微服务容器

设计洞察 · 学习目标

1. 用"独立 APK+platform 签名+ProxiedService（:40）"三要素复述 CarServiceImpl 的进程身份与崩溃隔离收益；2. 说出 CarServiceBase 接口三件套（init/release/dump）与书13 CoreStartable 的同构关系；3. 解释 CarFeatureController 按配置启用/禁用子服务的机制；4. 用 dumpsys car_service --list 列出当前设备实际装配的子服务清单

CarServiceImpl（:40 extends ProxiedService）是车机的"SystemServer"：启动时按 `CarFeatureController` 依次拉起各子服务（CarInputService:81、CarPowerManagementService:103、CarDrivingStateService、CarLocationService…）。每个子服务实现 `CarServiceBase` 接口（init/release/dump），**与书13 SystemUI 的 CoreStartable 同构**——大容器的"微服务注册制"。



图 1 AAOS 分层：App（car-lib）→ CarService（独立进程微服务容器）→ VHAL → 车辆总线，右侧为 OEM 五个定制点

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释车机 AAOS的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

> **设计洞察 · 逐段精读：CarService 与手机 SystemServer 的对照**
> CarService 是一个**普通 APK**（com.android.car 进程，platform 签名）——不是 system_server 的一部分。这与 SystemUI 的设计哲学一致：**产品特性子集独立进程**，崩溃不拖垮系统核心。手机没装 CarService APK=就是普通手机；装了=车机。

第二章 VHAL：车信号的统一语言

> **设计洞察 · 学习目标**
> 1. 把车速/档位/空调温度统一表述为 VehiclePropValue 四元组（prop+areaId+timestamp+status）；2. 解释 VehicleHal.java:71 的属性订阅/分发模型与手机 HAL 方法调用模型的本质差异；3. 追踪 PERF_VEHICLE_SPEED 从 CAN 信号到 CarDrivingStateService 再到 UI 限制的完整链；4. 用 cmd car_service get-carpropertyconfig 与 get-property-value 验证任一属性（附录 A）

VehicleHal（:71 implements HalClientCallback）是车机版 HAL：把 CAN 总线/车身控制器的物理信号抽象为 `VehiclePropValue`（属性 ID+值+时戳）——车速/档位/空调温度/方向盘转角/门开关……全部是属性。

text VHAL 结构（packages/services/Car/service/src/com/android/car/hal/）

```
VehicleHal.java:71（主入口，属性订阅/分发）
AidlVehicleStub.java（AIDL 后端，连接 vendor VHAL 进程）
VehiclePropConfig（属性元数据：类型/范围/权限）
→ Framework 侧消费：CarDrivingStateService（行驶状态）/
CarPowerManagementService（电源）/ CarInputService（方向盘按键）
// 讲解注释：VHAL 是书14 AIDL HAL 的车机版——
// vendor 侧实现 VehicleHal.aidl，框架侧 VehicleHal.java 是客户端。
```

面试高频：车速信号怎么变成"行驶中禁止操作视频"？VHAL 属性 PER Vehicle_SPEED → CarDrivingStateService 订阅 → 状态变化广播 → SystemUI/Policy 消费（车速>阈值=DRIVING → 限制 UI）。**这条链从物理信号到 UI 策略贯穿全书**。

第三章 车机电源状态机

设计洞察 · 学习目标

1. 背出 WAIT_FOR_VHAL→ON→SHUTDOWN_PREPARE→POST_SHUTDOWN/DEEP_SLEEP 主路径与 SHUTDOWN_CANCELLED 取消态；2. 解释熄火不等于关机的车规语义与 CAN 唤醒来源；3. 对照手机 PowerManagerService（书2）说出三点本质差异；4. 用 dumpsys car_service --services CarPowerManagementService 读出当前电源态

CarPowerManagementService（:103）与手机 PowerManagerService 的本质差异：

维度	手机	车机
关机语义	关机	**熄火**（深度睡眠但可被唤醒——开门/点火）
唤醒来源	闹钟/通知	**CAN 信号**（车门/点火/充电）
状态机	AWAKE/ASLEEP	**WAIT_FOR_VHAL→ON→SHUTDOWN_PREPARE→POST_SHUTDOWN_ENTER→…**
持久性	用户按键	车规：点火必须恢复到断前状态

设计洞察 · 设计动机：为什么车机不能"真的关机"

车门打开要立刻亮屏（安全法规）；熄火后仍需监控防盗/OTA——**"睡眠但可唤醒"是车规的硬要求**，与手机的"用户想开才开"完全不同。

第四章 多显示与乘客屏

设计洞察 · 学习目标

1. 解释仪表/中控/乘客屏各占一个 DisplayContent 的多屏模型（书5）与 per-display 焦点；2. 说明 CarActivityService 如何为每块屏维护独立 Activity 栈；3. 说出"乘客可视频、仪表禁止"安全策略的落点（UxR 按乘员区分发）；4. 结合第九章 OccupantZone 理解屏与座位的绑定关系

AAOS 多屏是标配：`CarActivityService` 管多 DisplayContent（书5 § 六）——仪表屏/中控主屏/乘客娱乐屏各独立 ActivityDisplay。乘客屏可看视频而仪表屏禁止（安全策略）。**书5 的 per-display 焦点与 13 的 PowerGroup 在此兑现**。

第五章 设计决策六问

设计洞察 · 学习目标

1. 不看书面独立复述六问的答案骨架；2. 为每问补一条源码证据（类名与行号，可引用第六～九章）；3. 把"独立进程/属性模型/电源独立链/ProxiedService"四组决策迁移评估到新硬件形态；4. 识别哪些决策由车规法规驱动、哪些是纯工程权衡

- **为什么 CarService 是独立进程而非 system_server 的一部分？** 崩溃隔离（车机服务死不拖系统）+ 车厂可独立定制（CarFeatureController 按 OEM 配置启用/禁用子服务）。
- **为什么 VHAL 是属性而非方法调用？** 车信号本质是"状态流"（速度连续变化）——订阅/推送模式天然匹配，方法调用不适合。
- **为什么车机电源不走手机 PowerManager？** 车规差异太大（熄火/唤醒语义/强制恢复）；VHAL 电源信号→CarPowerManagementService 是独立链路，与手机 PMS 平行不合并。
- **为什么 CarService 用 ProxiedService 而非直启？** 按需懒加载：用户没进入车机功能时不启动全部子服务（省启动时间/内存）。
- **为什么 AAOS 与 Android Auto 是两个产品？** AAOS=操作系统（整机车机）；Auto=投射（手机投到车屏）——**一个是 OS，一个是 App**。
- **为什么车机多屏默认禁乘客看仪表？** 安全法规：驾驶员分心=违法——多屏策略的安全底线。

附录 A 观测与实战

目标	命令	看点
车机全表	adb shell dumpsys car_service	子服务状态
VHAL 属性	adb shell cmd car_service get-property-value 0x11600207	车速等
电源状态	dumpsys car_service power	当前态/历史
驾驶状态	dumpsys car_service driving-state	行驶/驻车

附录 B 快问快答（12 条）

- AAOS 与 Android Auto 区别？——OS 整车 vs 手机投射 App。
- CarServiceImpl 什么身份？——ProxiedService（:40），微服务容器。
- VHAL 是什么？——车信号 HAL（VehicleHal:71，属性订阅/分发）。
- 车速→禁止视频的链？——VHAL→CarDrivingStateService→Policy→UI。

- 车机电源与手机差异？——熄火≠关机；CAN 唤醒；强制恢复。
- 多屏怎么管？——CarActivityService+多 DisplayContent（书5）。
- 车机 CarService 是什么进程？——com.android.car（独立 APK）。
- 车机能用手机 App 吗？——部分兼容（但 UI 需适配车屏）。
- VHAL 走什么 IPC？——AIDL HAL（书14 模板）。
- 乘客屏与主屏差异？——per-display 策略（乘客可视频，仪表禁止）。
- 车规安全怎么落地？——DrivingState 限制+多屏策略+输入拦截。
- 车机 CTS 与手机一样吗？——加 CTS-Auto（车机专属用例）。

附录 C 自检题

- VHAL 属性模型与手机 HAL 方法调用的适用场景差异。
- 车机电源七态与手机四态的映射/新增态。
- 车速信号到 UI 策略限制的完整链路。

第六章 启动链：CarServiceImpl 到 ICarImpl.init 的微服务装配

> **设计洞察 · 学习目标**
> 1. 复述 SystemServer 拉起 com.android.car 后到 addService("car_service") 的六步路径并标出唯一的跨进程注册点；2. 解释"构造期依赖注入+init 期统一通电"两阶段设计的动机；3. 说出 mAllServices 装配顺序背后的依赖规则并举例（CarPropertyService 必须先于 CarUxRestrictionsService）；4. 用 dumpsys car_service --list 与 CAR.InitTiming trace 逐服务归因启动耗时

这一段在整条链路中的位置：第一章回答了"CarService 是谁"（独立 APK 的微服务容器），本章回答"容器是怎么装满的"。全链只有**一次跨进程注册**：CarServiceImpl.onCreate（CarServiceImpl.java:52）末尾的 `ServiceManagerHelper.addService("car_service", mICarImpl)`（:77）——此前一切发生在 com.android.car 进程内，此后 App 侧 `Car.createCar` 才有东西可绑（该时序见第八章 8.4）。链路为：SystemServer 拉起 com.android.car（普通 APK，非 critical 进程，书2 对照）→ CarServiceImpl.onCreate（:52）→ VehicleStub.newVehicleStub（:58，AIDL 后端探测失败自动回退 HIDL，VehicleStub.java:76）→ new ICarImpl（:68，构造期完成 30 余个子服务的依赖注入）→ mICarImpl.init()（:73，统一"通电"）→ addService 注册（:77）→ 置属性 boot.car_service_created=1（:78）。与书13 SystemUI 的 CoreStartable 总线对照着读：同构，但 CarService 多了一个"先连 VHAL"的前置步骤——所有车信号都从这一步开始有了源头。

6.1 onCreate 逐段精读：连 VHAL、造容器、注册 binder

java packages/services/Car/service/src/com/android/car/CarServiceImpl.java:51-83（骨架+讲解注释）

```
@Override
public void onCreate() {
// 讲解注释①：LimitedTimingsTraceLog 只刷超过阈值的片段
//  （CAR_SERVICE_INIT_TIMING_MIN_DURATION_MS=5，:42）——
// perfetto 按 CAR.InitTiming 标签过滤即可切出启动瀑布图
LimitedTimingsTraceLog initTiming = new LimitedTimingsTraceLog(CAR_SERVICE_INIT_TIMING_TAG,
TraceHelper.TRACE_TAG_CAR_SERVICE, CAR_SERVICE_INIT_TIMING_MIN_DURATION_MS);
initTiming.traceBegin("CarService.onCreate");
initTiming.traceBegin("getVehicle");
mVehicle = VehicleStub.newVehicleStub(); // 讲解注释②：连 VHAL 第一步——
initTiming.traceEnd(); // "getVehicle" // AIDL 探测失败自动回退 HIDL
EventLogHelper.writeCarServiceCreate(/* hasVhal= */ mVehicle.isValid());
mVehicleInterfaceName = mVehicle.getInterfaceDescriptor();
Slogf.i(CarLog.TAG_SERVICE, "Connected to " + mVehicleInterfaceName);
EventLogHelper.writeCarServiceConnected(mVehicleInterfaceName);
// 讲解注释③：两阶段的第一阶段——只"构造"不"启动"。构造器里
// new 出 VehicleHal 与 30 余个子服务对象并完成依赖注入
//  （谁持谁、初始化顺序靠前靠后），但没有任何 init() 执行
mICarImpl = new ICarImpl(this,
getBuiltinPackageContext(),
mVehicle,
SystemInterface.Builder.defaultSystemInterface(this).build(),
mVehicleInterfaceName);
mICarImpl.init(); // 讲解注释④：第二阶段——统一"通电"（见 6.2）
// 讲解注释⑤：给 VHAL 进程挂死亡回调——vendor 进程一死，
// serviceDied（:123）里 Process.killProcess 自杀，让 init
// 重建整个 CarService："HAL 死了框架陪葬"的强一致策略
mVehicle.linkToDeath(mVehicleDeathRecipient);
// 讲解注释⑥：全链唯一一次跨进程注册——此后 ServiceManager 里
// 才有 "car_service"，App 侧 ServiceManagerHelper 查询才拿得到句柄
ServiceManagerHelper.addService("car_service", mICarImpl);
SystemPropertiesHelper.set("boot.car_service_created", "1"); // 讲解注释⑦
super.onCreate();
initTiming.traceEnd(); // "CarService.onCreate"
}
```

四点值得抠住不放。**其一**，onCreate 里先连 VHAL 再造 ICarImpl：`mVehicle.isValid()` 决定 hasVhal 事件位，VHAL 不在时 CarService 依然会启动（只是没有车信号），这是"调试便利性优先于完整性"的选择。**其二**，注释③④的两阶段是本章主线——构造器与 init 的分工不是历史包袱，而是"依赖图构建"与"依赖图通电"的显式分离，6.3 的装配顺序只有在两阶段分离后才可能表达。**其三**，注释⑤的死亡陪葬是车机特有的强一致：VHAL 是车信号的唯一事实来源，半死状态（部分属性有效）比全死更危险。**其四**，`boot.car_service_created=1`（注释⑦）是跨进程握手点——车机上依赖 CarService 的组件（如早期 VHAL 客户端）轮询这个属性而不是盲等 binder。

6.2 ICarImpl.init：先通 VHAL，再逐个"点亮"子服务

java packages/services/Car/service/src/com/android/car/ICarImpl.java:471-497（骨架+讲解注释）

```
@MainThread
void init() {
LimitedTimingsTraceLog t = new LimitedTimingsTraceLog(CAR_SERVICE_INIT_TIMING_TAG,
TraceHelper.TRACE_TAG_CAR_SERVICE, CAR_SERVICE_INIT_TIMING_MIN_DURATION_MS);
t.traceBegin("ICarImpl.init");
t.traceBegin("VHAL.init");
mHal.init(); // 讲解注释①：先"通电"VHAL——拉全量属性表、
t.traceEnd(); // 建立属性→HAL 服务的分发表（第七章 7.3 逐行走读）。
// 此后任何子服务 init 时才有车信号可订阅
t.traceBegin("CarService.initAllServices");
for (CarServiceBase service : mAllServices) {
t.traceBegin(service.getClass().getSimpleName());
service.init(); // 讲解注释②：严格按构造期 mAllServices 声明
t.traceEnd(); // 顺序串行 init——顺序即依赖（见 6.3 表）；
} // trace 切片名=类名，可逐服务归因耗时
t.traceEnd(); // "CarService.initAllServices"
t.traceEnd(); // "ICarImpl.init"
}
void release() {
// 讲解注释③：释放严格逆序——"依赖者先死，被依赖者后死"。
// 例如 CarUxRestrictionsService 依赖 CarDrivingStateService，
// 前者装配靠后、释放先走，不会出现悬空引用（栈展开思想）
for (int i = mAllServices.length - 1; i >= 0; i--) {
mAllServices[i].release();
}
mHal.release(); // 讲解注释④：HAL 客户端最后释放——它被所有人依赖
}
```

注意 init() 标了 `@MainThread`——CarServiceImpl 是个 Service，onCreate 跑在主线程，所以这条装配链全程在 com.android.car 的主线程上串行执行。这解释了为什么 CarService 启动慢会直接拖慢开机（车机的"开机完成"通常以 boot.car_service_created 与 boot_complete 一起为信号）：任何一个子服务的 init 卡住，整个链就停在那里，trace 里的切片名（注释②）就是定位卡点的第一入口。

6.3 装配顺序即依赖顺序：mAllServices 的声明学

构造器末尾有一句总纲（ICarImpl.java:416）：**"Be careful with order. Service depending on other service should be inited later."**（顺序要小心：依赖其他服务的服务应更晚 init）。随后 40 行 `allServices.add(...)` 就是把这句话落成代码，声明处的行内注释直接写明依赖关系：

装配序	子服务	声明处依赖注释（源码原文）
1	CarFeatureController	功能开关先行，决定后续哪些子服务要构造
2	CarPropertyService	ICarImpl.java:419 注明：mCarUXRestrictionsService depends on it

装配序	子服务	声明处依赖注释（源码原文）
3	CarOccupantZoneService	ICarImpl.java:420 注明：mCarUxRestrictionsService depends on it
4	CarUxRestrictionsManagerService	依赖上面两个（构造器实参即证据，:249）
5	CarUserService	依赖 UxR（驾驶中限制用户操作）
6	SystemActivityMonitoringService、CarPowerManagementService、CarDrivingStateService	电源链与驾驶状态链
…	CarPackageManagerService、CarInputService、CarAudioService、CarMediaService…	常规产品子服务
末	CarActivityService、ExperimentalFeatureServiceController	ICarImpl.java:455 注明后者永远最后

> **设计洞察 · 设计洞察：为什么"构造期注入、init 期通电"而不是一步到位**
> 一步到位（构造即 init）会在构造器里引发连锁 init：CarUxRestrictionsService 的构造函数里去摸 CarDrivingStateService 的运行态，而后者可能还没准备好——顺序错误从"运行期 bug"升级为"构造期崩溃"。两阶段把顺序约束收敛到一个循环（6.2 注释②）：构造期只要求"被依赖者的引用已存在"（编译器/构造器实参强制保证），运行期顺序只看 mAllServices 一张表。改造这个文件时，新子服务插错位置不会编译报错，但会在 init 或首次使用时炸——所以源码用行内注释+@VisibleForTesting 构造器双重防呆，读代码时先看注释再看插队位置。

> **设计洞察 · 交叉对照：CarService 装配与书13 SystemUI 启动总线**
> 同构点：都是"接口三件套（init/onStart、release、dump）+确定性顺序+trace 逐组件归因"；差异点有三——SystemUI 用 Dagger @IntoMap 在**编译期**生成组件表，CarService 用手写 ArrayList 在**源码期**声明顺序；SystemUI 的组件间几乎不依赖（事件总线解耦），CarService 的子服务是显式依赖链（电源/驾驶状态/属性三类公共底座）；SystemUI 崩了 init 拉起重来即可，CarService 崩溃重连 VHAL 后还要重放电源状态机（第三章）——所以本章的 release 逆序释放（6.2 注释③）在 SystemUI 里找不到对应物。

回到全链：addService 完成后，App 侧才能建立连接（第八章），VHAL 才有消费方（第七章），多屏与用户绑定才有载体（第九章）。附录 A 的 `dumpsys car_service --list` 打出的就是这张装配表的运行时实况。

第七章 VHAL 属性模型：prop 位结构、areaId 与分发表

设计洞察 · 学习目标

1. 把任意属性 ID 按位分解为 group/area/type/index 四段（如 0x11600207）；2. 区分 VehiclePropertyChangeMode 三种取值与"能否订阅"的关系；3. 解释 areaId 的语义：全局区 vs 区域区（SEAT/WINDOW/DOOR），并说出 AreaID 位掩码映射规则；4. 读懂 VehiclePropConfig 各字段并知道 VHAL init 时如何把属性分派给 9 个 HalServiceBase

这一段在整条链路中的位置：第二章建立了"一切车信号皆属性"的直觉，本章下钻到属性模型本体——它定义在 `hardware/interfaces/automotive/vehicle/2.0/types.hal`（HIDL 契约，vendor 侧按此实现，书14 模板），被框架侧 VehicleHal.java 消费。一条车信号要过三道模型关：**ID 层**（prop 的位结构，:60-98 注释块定义 AreaID 规则）→ **配置层**（VehiclePropConfig:4019，属性的静态能力表）→ **值层**（VehiclePropValue:4065，timestamp+areaId+prop+status+RawValue）。本章走读配置层与框架侧的分发表构建；订阅与回调的动态过程留给第八章。

7.1 prop 的位结构：0x11600207 是什么

属性 ID 不是黑盒数字，而是四个字段的**位或打包**（types.hal:144-153 注释：每个属性必须由 group+area+type+唯一 id 组成）：

位段	掩码	含义	典型取值
GROUP	0xf0000000	谁声明	SYSTEM=0x10000000（:130）/VENDOR=0x20000000（:136）
AREA	0x0f000000	作用区类型	GLOBAL=0x01000000/SEAT=0x05000000/WINDOW=0x03000000/DOOR=0x06000000（:99-114）
TYPE	0x00ff0000	值的数据类型	INT32=0x00400000/FLOAT=0x00600000/BOOLEAN=0x00200000/MIXED=0x00e00000（:10-56）
INDEX	0x0000ffff	属性编号	0x0100-0xffff 自定义段（:149）

拿车速验算：PERF_VEHICLE_SPEED = 0x0207 | VehiclePropertyGroup:SYSTEM | VehiclePropertyType:FLOAT | VehicleArea:GLOBAL（types.hal:367-371）= 0x10000000+0x01000000+0x00600000+0x207 = **0x11600207**——这正是附录 A 里 `cmd car_service get-property-value 0x11600207` 查的那个数。面试里写得出这个分解，VHAL 的门槛就算过了。

> **设计洞察 · 逐段精读：为什么用位打包而不用字符串或枚举类**
> 三重收益：**二进制兼容**——vendor VHAL 与框架可能由不同团队、不同版本各自编译，位段设计让"新增属性"不破坏旧解析（老框架遇到不认识的 AREA 位段直接拒绝即可）；**掩码运算即查询**——判断"这是不是个 SEAT 属性"一次与运算（prop & 0x0f000000 == 0x05000000），配表、路由、dump 全靠这个；**VTS 可校验**—— 位结构是机器可检查的强规则，VTS 测试能自动发现 vendor 违规打包。代价是可读性差（0x11600207 谁也认不出来），所以 car-lib 里有完整的 VehiclePropertyIds 常量表帮 App 隔离原始数字。

7.2 VehiclePropConfig：属性的静态能力表

text hardware/interfaces/automotive/vehicle/2.0/types.hal:4019-4047（骨架+讲解注释）

```
struct VehiclePropConfig {
/** Property identifier */
int32_t prop; // 讲解注释①：属性 ID（7.1 的位结构），
// vendor 与框架据此对齐一切
/**
* Defines if the property is read or write or both.
*/
VehiclePropertyAccess access; // 讲解注释②：访问权 NONE/READ/WRITE/READ_WRITE
//  （:3888）——框架按它配权限与 UI 灰显
/**
* Defines the change mode of the property.
*/
VehiclePropertyChangeMode changeMode; // 讲解注释③：变化模式——决定"能否订阅"
// 与"订阅要不要采样率"（8.2 详述）
/**
* Contains per-area configuration.
*/
vec<VehicleAreaConfig> areaConfigs; // 讲解注释④：区域配置数组——全局属性
// 通常一项（areaId 被忽略，:3997），
// 区域属性每区一项、各带合法值域
//  （min/maxInt32/Int64/Float，:4005-4014）
/** Contains additional configuration parameters */
vec<int32_t> configArray; // 讲解注释⑤：附加参数——如 MIXED 类型的
// 载荷结构描述（:34-48 的格式约定）
/**
* Some properties may require additional information passed over this
* string. Most properties do not need to set this.
*/
string configString; // 讲解注释⑥：少数字符串参数（如 OTA 校准串）
/**
* Min sample rate in Hz.
* Must be defined for VehiclePropertyChangeMode::CONTINUOUS
*/
float minSampleRate; // 讲解注释⑦：采样率上下界——只对 CONTINUOUS
// 有意义；订阅时请求率被夹到
// [minSampleRate, maxSampleRate] 区间
/**
* Must be defined for VehiclePropertyChangeMode::CONTINUOUS
* Max sample rate in Hz.
*/
float maxSampleRate; // 讲解注释⑧：——CAN 总线带宽有限，
```

```
// vendor 用它宣示物理信号的真实频率上限
};
```

七号注释值得展开：areaConfigs 是 areaId 模型的落点。VehicleAreaConfig（:3995）里每项一个 areaId（如 SEAT_ROW_1_LEFT=0x00000001，VehicleAreaSeat:3942 起）加该区的合法值域——**同一属性在不同区可以有不同的合法范围**（主驾座椅加热 0-3 档、副驾只有 0-1 档，就是两个 areaConfig 的事）。而"哪些区合成一个 areaId"有铁律（types.hal:77-86 注释原文的翻译）：**一个 AreaID 内的多个区必须永远同时变化、不可独立控制；数组中任意两个 AreaID 必须可独立控制；一个区只能出现在一个 AreaID 里**。这把"物理耦合"翻译成了"数据结构"——vendor 报什么 areaId 划分，框架就按什么粒度控制。

> **设计洞察 · 设计洞察：status 字段是车规可靠性的喉舌**
>
> VehiclePropertyStatus（:3899）三值：AVAILABLE/UNAVAILABLE/ERROR。注释里写得极严：UNAVAILABLE 是"暂时不可用"（如传感器掉线），**禁止**用来表达"这台车根本没有的功能"；set 一个 UNAVAILABLE 的属性 MAY 返回 NOT_AVAILABLE；HAL 收到 Android 写来的值时**必须忽略值里的 status 位**——否则 App 伪造一个 ERROR 就能让真车信号被判死。对照手机的"传感器没有就别注册"模型：车信号永远在线，只是健康度在波动，所以健康度必须是值的一部分而不是配置的一部分。

7.3 VehicleHal.init：全量配置表与属性分发表

框架侧消费这份契约的地方在 VehicleHal（第二章已介绍其身份）：init 时拉全量配置、把每个属性**指派给恰好一个** HalServiceBase：

java packages/services/Car/service/src/com/android/car/hal/VehicleHal.java:218-259（骨架+讲解注释）

```
public void init() {
fetchAllPropConfigs(); // 讲解注释①：HalClient.getAllPropConfigs()（:191）
// 一次性拉全量 HalPropConfig 存进 mAllProperties；
// 失败直接抛 RuntimeException——没有配置表，
// 整个 CarService 无从工作，宁可快速失败
// PropertyHalService will take most properties, so make it big enough.
ArrayMap<HalServiceBase, ArrayList<HalPropConfig>> configsForAllServices;
synchronized (mLock) {
configsForAllServices = new ArrayMap<>(mAllServices.size());
for (int i = 0; i < mAllServices.size(); i++) {
ArrayList<HalPropConfig> configsForService = new ArrayList();
HalServiceBase service = mAllServices.get(i); // 讲解注释②：9 个 HAL
// 服务（构造器 :166-176）
configsForAllServices.put(service, configsForService);
int[] supportedProps = service.getAllSupportedProperties();
if (supportedProps.length == 0) {
for (int j = 0; j < mAllProperties.size(); j++) {
Integer propId = mAllProperties.keyAt(j);
if (service.isSupportedProperty(propId)) { // 讲解注释③：逐个问询——
HalPropConfig config = mAllProperties.get(propId);
mPropertyHandlers.append(propId, service); // 无预声明清单的
configsForService.add(config); // HAL 服务用谓词认领
}
}
} else {
```

```
for (int prop : supportedProps) {
HalPropConfig config = mAllProperties.get(prop);
if (config == null) {
continue; // 讲解注释④：vendor 没报的属性直接跳过——
} // 声明了不代表真车支持，一切以配置表为准
mPropertyHandlers.append(prop, service); // 讲解注释⑤：核心产物——
configsForService.add(config); // propId→服务 的路由表，
} // 回调分发全靠它（8.3）
}
}
}
for (Map.Entry<HalServiceBase, ArrayList<HalPropConfig>> entry
: configsForAllServices.entrySet()) {
HalServiceBase service = entry.getKey();
ArrayList<HalPropConfig> configsForService = entry.getValue();
service.takeProperties(configsForService); // 讲解注释⑥：把自己名下的配置
service.init(); // 发给各 HAL 服务，再逐个 init——
} // PowerHal/PropertyHal… 由此各就各位
}
```

三个 HalServiceBase 的分工决定了"为什么有这一层"：PowerHalService 只认电源属性（AP_POWER_STATE_REQ 等，第三章）、InputHalService 只认方向盘/旋钮按键、PropertyHalService 认领**其余几乎所有**属性并桥接给 CarPropertyService——VehicleHal.java:174 的注释说得很直白：**"mPropertyHal must be the last so that on init/release it can be used for all other HAL services properties"**（PropertyHal 必须最后，init/release 时其他 HAL 服务的属性都要过它的手）。这就是 6.2 "先通 VHAL"注释①的另一半：通电不只是连上 vendor，还包括把 9 个认领者排好座次。

> **设计洞察 · 设计洞察：一层"属性路由"买到了什么**
>
> VHAL 框架侧没有把每个功能做成一个大 if-else，而是把"属性归谁"做成表（mPropertyHandlers），把"事件走哪"做成同表的逆查询（8.3 的 onPropertyEvent 查它找目标服务）。新增一类车信号（如胎压）= 新增一个 HalServiceBase 声明认领+init 订阅，VehicleHal 主干零改动——这是**开闭原则在 HAL 层的样子**。对照书13 的 CoreStartable：同样是"接口+注册表"吃掉扩展性，只是那里的注册表在 Dagger 图里，这里的注册表在 mPropertyHandlers 的 SparseArray 里。vendor 侧同理：default 实现（hardware/interfaces/automotive/vehicle/2.0/default/，AIDL 版在 aidl/impl/）提供了 reference 骨架，OEM 只填属性实现。

回到全链：本章是静态模型（ID/配置/路由表），第八章让模型动起来——订阅、采样率协商与事件回调的完整闭环。

第八章 属性订阅链：registerListener 到 onPropertyEvent 的完整闭环

设计洞察 · 学习目标

1. 画出 App 注册监听到收到回调的十级链路并标注每次线程/binder 切换；2. 解释采样率协商规则（多客户端取最大、被夹到 config 区间）与初始值补发机制；3. 说明 VehicleHal.onPropertyEvent 为何"锁内收集、锁外分发"以及 CarPropertyService 为何在锁外回调；4. 复述 Car.createCar 的连接时序（轮询/绑定/回调三路）与断连时的 Manager 失效处理

这一段在整条链路中的位置：第六章让 CarService"通电"、第七章建好属性模型与路由表，本章让车信号真正流到 App。下行（订阅）链：App `CarPropertyManager.registerCallback` → binder → CarPropertyService.registerListener（CarPropertyService.java:245）→ PropertyHalService.subscribeProperty（PropertyHalService.java:301，管理属性→VHAL 属性换算）→ VehicleHal.subscribeProperty（VehicleHal.java:355）→ HalClient.subscribe（HalClient.java:99）→ vendor VHAL 进程。上行（事件）链：vendor 回调 onPropertyEvent → VehicleHal.onPropertyEvent（:663，专用的 VehicleHal HandlerThread）→ PropertyHalService.onHalEvents（:408）→ CarPropertyService.onPropertyChange（:622，实现 PropertyHalListener 接口，:59）→ Client.onEvent（:178，oneway binder）→ App 回调线程。全程两次 binder：App↔CarService 一次，CarService↔vendor HAL 一次（书3 的 binder 语义 + 书14 的 AIDL HAL 模板在此会师）。

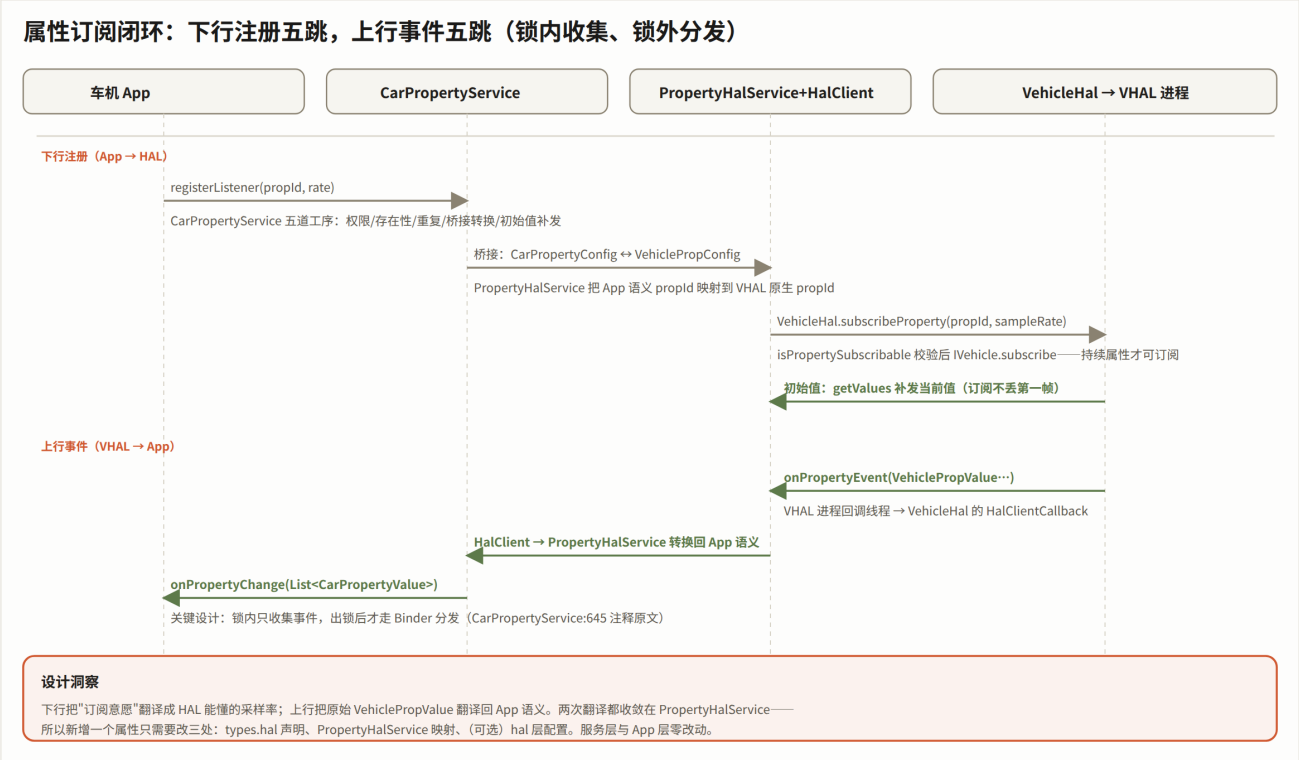


图 2 属性订阅闭环：下行注册五跳与上行事件五跳，两次翻译都收敛在 PropertyHalService

8.1 下行链精读：registerListener 的五道工序

java packages/services/Car/service/src/com/android/car/CarPropertyService.java:245-307（骨架+讲解注释）

```
public void registerListener(int propId, float rate, ICarPropertyEventListener listener)
```

```
throws IllegalArgumentException {
if (listener == null) {
Slogf.e(TAG, "registerListener: Listener is null.");
throw new IllegalArgumentException("listener cannot be null."); // 讲解注释①
}
IBinder listenerBinder = listener.asBinder();
CarPropertyConfig propertyConfig;
Client finalClient;
synchronized (mLock) {
propertyConfig = mConfigs.get(propId);
if (propertyConfig == null) {
// Do not attempt to register an invalid propId
Slogf.e(TAG, "registerListener: propId is not in config list: 0x"
+ toHexString(propId));
return; // 讲解注释②：配置表没有的属性静默拒绝——
} // 不支持≠出错，App 端应先查 getCarPropertyConfig
CarServiceUtils.assertPermission(mContext, mHal.getReadPermission(propId));
// 讲解注释③：权限闸——每个属性在框架侧有独立读权限
//  （VehiclePropConfig.access 的框架映射，7.2 注释②）
// Get or create the client for this listener
Client client = mClientMap.get(listenerBinder);
if (client == null) {
client = new Client(listener); // 讲解注释④：一个 App 进程一个 Client——
if (client.isDead()) { // 以 binder 为键去重；死链接端直接回绝
Slogf.w(TAG, "the ICarPropertyEventListener is already dead");
return;
}
mClientMap.put(listenerBinder, client);
}
client.addProperty(propId, rate);
// Insert the client into the propId --> clients map
List<Client> clients = mPropIdClientMap.get(propId);
if (clients == null) {
clients = new ArrayList<Client>();
mPropIdClientMap.put(propId, clients);
}
if (!clients.contains(client)) {
clients.add(client); // 讲解注释⑤：双表记账——propId→clients
} // 供上行分发（8.3），client→props 供注销
// Set the new rate
if (rate > mHal.getSampleRate(propId)) {
mHal.subscribeProperty(propId, rate); // 讲解注释⑥：采样率协商——只有
} // 新请求率高于当前率才重新订阅，
// 多客户端永远取最大值
finalClient = client;
}
// propertyConfig and client are NonNull.
mHandler.post(() ->
getAndDispatchPropertyInitValue(propertyConfig, finalClient));
// 讲解注释⑦：初始值补发——注册完成后立刻把当前值推一
// 遍给新客户端（:309），App 无需再手动 get 一轮；
// post 出锁是为了注册路径不被慢速 get 拖死
}
```

六号注释是整章最容易被面试考到的点：CONTINUOUS 属性（车速是典型）的物理信号频率由 vendor 决定（7.2 注释⑦的 min/maxSampleRate），N 个 App 各报一个期望率时框架**取最大**再向 VHAL 重订阅——"最挑剔的听者决定广播频率"。代价是任何人升频、全体付账（第六章习题的订阅风暴即源于此）。注销侧（unregisterListenerBinderForProps，:169 与 :347）做对称操作：最后一个监听者离开才真正 unsubscribe。

> **设计洞察 · 逐段精读：初始值补发的两难**
>
> 注释⑦解决了"订阅与事件间隙丢首帧"——从注册成功到 vendor 下一条 ON_CHANGE 事件可能隔几分钟（如门开关），不补发 App 会停在"未知态"。但补发本身有竞态：get 出来的初始值与补发途中到达的新事件可能乱序。框架的取舍是"宁可短暂旧值，不做序列化保证"——CarPropertyValue 自带 timestamp（7.1 值层），严谨的 App 用时戳丢弃过期帧。这解释了官方文档里"回调可能收到注册前的值"这句看似反直觉的话。

8.2 上行链精读：一次事件从 vendor 到 App 的五跳

先看路由表的消费端——VehicleHal.onPropertyEvent。HalClient 内部把 vendor 的 binder 回调 post 到 VehicleHal 专属 HandlerThread（HalClient.java:237 起的 CallbackHandler），然后：

java packages/services/Car/service/src/com/android/car/hal/VehicleHal.java:655-689（骨架+讲解注释）

```
static boolean isPropertySubscribable(HalPropConfig config) {
if ((config.getAccess() & VehiclePropertyAccess.READ) == 0
|| (config.getChangeMode() == VehiclePropertyChangeMode.STATIC)) {
return false; // 讲解注释①：订阅资格闸——不可读的不能订；
} // STATIC 属性永不变化（7.2 表），订了也白订
return true;
}
private final ArraySet<HalServiceBase> mServicesToDispatch = new ArraySet<>();
// should be posted to the mHandlerThread
@Override
public void onPropertyEvent(ArrayList<HalPropValue> propValues) {
synchronized (mLock) {
for (int i = 0; i < propValues.size(); i++) {
HalPropValue v = propValues.get(i);
int propId = v.getPropId();
HalServiceBase service = mPropertyHandlers.get(propId);
if (service == null) {
Slogf.e(CarLog.TAG_HAL, "HalService not found for prop: 0x"
+ toHexString(propId));
continue; // 讲解注释②：7.3 路由表的反查——查不到说明
} // vendor 发了个没人认领的属性，丢事件只记日志
service.getDispatchList().add(v);
mServicesToDispatch.add(service); // 讲解注释③：按服务聚批——
VehiclePropertyEventInfo info = mEventLog.get(propId);
if (info == null) {
info = new VehiclePropertyEventInfo(v);
mEventLog.put(propId, info);
} else {
info.addNewEvent(v); // 讲解注释④：每属性事件记账——dumpsys
} // --hal 时"该属性多久没来事件"就看这里
}
```

```
}
for (HalServiceBase s : mServicesToDispatch) {
s.onHalEvents(s.getDispatchList()); // 讲解注释⑤：锁外分发——同一服务的
s.getDispatchList().clear(); // 一批事件一次给全，下游免锁竞争
}
mServicesToDispatch.clear();
}
```

第二跳是 PropertyHalService.onHalEvents（PropertyHalService.java:408-441）：把 VHAL 属性 ID 换算回管理属性 ID（`halToManagerPropId`，框架对 App 屏蔽了部分内部属性）、debug 版做载荷合法性抽查（checkPayload）、再 `v.toCarPropertyValue(mgrPropId, propConfig)` 转成 App 世界的 CarPropertyValue，聚批后调 `listener.onPropertyChange(...)`——listener 就是 CarPropertyService（第六章 init 时 `mHal.setListener(this)` 注册，CarPropertyService.java:198）。第三跳 CarPropertyService.onPropertyChange（:622）查 8.1 注释⑤的 propId→clients 表按客户端聚批。源码里有一段必须原样记住的设计注释（:645-648）：**回调必须在锁外发——onEvent 是 oneway binder，若在锁内调用，回程（如客户端同步 unregister）会重入 mLock 造成死锁**。第四跳 Client.onEvent（:178）oneway 出进程，第五跳 App 侧回调线程拿到 CarPropertyEvent 列表。

> **设计洞察 · 设计洞察：两级"聚批+锁外分发"是同一个模式的两次套用**
>
> VehicleHal 层把一批原始值按 HalServiceBase 聚批，CarPropertyService 层把一批 CarPropertyValue 按 Client 聚批——都在锁内做表操作、锁外做分发。动机一致：binder/回调是慢操作且可能重入，持锁分发会把"上游吞吐"耦合到"下游速度"上。对照书13 通知管线的 GroupCoalescer（合批器）：那里是为了 UI 稳定性合并通知事件，这里是为了锁纪律合并 HAL 事件；同一思想在不同层面各长了一副面孔，串起来读是丛书设计的题眼。

8.3 App 侧入口：Car.createCar 的连接时序

car-lib 的 Car（car-lib/src/android/car/Car.java）是 App 拿到一切 Car*Manager 的门。createCar（:1624 的四参版本是全功能形态）同时开三路：

路径	机制	源码锚点
轮询	50ms 一次查 ServiceManager 里 car_service 是否就绪（开机早期服务未注册时）	Car.java:1220 间隔常量，:1644 循环体
绑定	startCarService 绑定 com.android.car/.CarService	Car.java:1661 起的 startCarService 路径
回调	就绪后 dispatchCarReadyToMainThread 或 ServiceConnection.onServiceConnected 通知 CarServiceLifecycleListener	Car.java:1312-1328

断连路径同样三路收口：onServiceDisconnected（:1341-1353）→ `mFeatures.resetCache()`（CarService 可能因 OEM 特性开关重启，缓存必须作废）→ handleCarDisconnectLocked（:1786）把 mService 置空、mConnectionRetryCount 归零、tearDownCarManagersLocked 让所有已发

Manager 失效——这就是官方要求 App 持有 CarServiceLifecycleListener 并在 onLifecycleChanged(false) 时停用 Manager 的源码根据。对照第六章的 VHAL 死亡陪葬：App 侧看到的"CarService 重启"就是一次 disconnect→reconnect 循环，监听器写得健壮与否，直接决定车机上"信号断一小会"还是"App 半残到下次点火"。

> **设计洞察 · 交叉对照：本章链路与书3 Binder 十跳**
> 下行链的 binder 跳点：App→CarService 一次（registerListener，传入的是 App 的 binder 端点）；CarService→vendor VHAL 一次（subscribe，AIDL HAL，书14 模板）。上行链方向相反但对称：vendor→CarService（onPropertyEvent）、CarService→App（Client.onEvent）。加上 ServiceManager 查询与 linkToDeath 注册，一条完整的车信号链是 6 次 binder 交互——比手机上一个普通系统服务调用重一倍，这是"信号频率×跨进程成本"的车机税，也是 8.1 注释⑥采样率协商要省的钱。

回到全链：到此，车速信号已经能流到任何 App。下一个问题是"哪个座位上的人看到哪个信号"——多屏、多用户与音区的三角绑定，第九章。

第九章 CarOccupantZoneService：座位-屏幕-用户的三角绑定

> **设计洞察 · 学习目标**
> 1. 复述乘员区三要素（occupantZoneId/occupantType/座位位形）与资源配置字符串格式；2. 解释 driverSeat 如何驱动左右舵判定并影响全部 zone 计算；3. 追踪 displayId 与 userId 如何经 OccupantZone 双向查询；4. 说明 audio zone 与 occupant zone/user 的绑定链，并知道两条 shell 命令能在实车上重绑

这一段在整条链路中的位置：第四章说了"AAOS 多屏是标配"，本章回答多屏背后的**归属**问题：哪块屏属于哪个座位、哪个用户登录在哪个座位、哪个音区为谁出声。CarOccupantZoneService（CarOccupantZoneService.java:79）在装配表里排在第 3 位（第六章 6.3 表，被 CarUxRestrictionsService 依赖），init（:271）时完成三角绑定：**座位位形**（从 VHAL 的 INFO_DRIVER_SEAT 读驾驶座，经 CarInfoManager:274）→ **区-屏映射**（解析 OEM 资源 config_occupant_zones:937 与 config_occupant_display_mapping:1047）→ **区-用户映射**（handleUserChangesLocked:283，随用户切换事件动态维护）。CarAudioService 再把音频路由挂到这棵树上（9.3）。它同时是 App 侧 API（CarOccupantZoneManager，常量 :72/:112/:117/:121）与服务端事实表。

9.1 init 逐段精读：从驾驶座开始的一切

java packages/services/Car/service/src/com/android/car/CarOccupantZoneService.java:271-358（骨架+讲解注释）

```
public void init() {
// This does not require connection as binder will be passed directly.
Car car = new Car(mContext, /* service= */ null, /* handler= */ null);
CarInfoManager infoManager = new CarInfoManager(car, CarLocalServices.getService(
CarPropertyService.class)); // 讲解注释①：进程内直连——装配表里
```

```
// CarPropertyService 排第 2，此刻必然就绪
int driverSeat = infoManager.getDriverSeat(); // 讲解注释②：驾驶座位形来自
// VHAL 属性 INFO_DRIVER_SEAT
//  （SEAT 区属性，7.1 的 AREA 位段）
synchronized (mLock) {
mDriverSeat = driverSeat;
parseOccupantZoneConfigsLocked(); // 讲解注释③：解析区定义（座位+类型）
// ——9.2 逐段走读
parseDisplayConfigsLocked(); // 讲解注释④：解析区-屏映射
//  （config_occupant_display_mapping）
handleActiveDisplaysLocked(); // 讲解注释⑤：把热插拔前的物理屏
// 对号入座到各 zone
handleAudioZoneChangesLocked(); // 讲解注释⑥：音区跟随（9.3）
handleUserChangesLocked(); // 讲解注释⑦：用户落位（9.1 末段）
}
CarUserService userService = CarLocalServices.getService(CarUserService.class);
UserLifecycleEventFilter userSwitchingEventFilter = new UserLifecycleEventFilter.Builder()
.addEventType(USER_LIFECYCLE_EVENT_TYPE_SWITCHING).build();
userService.addUserLifecycleListener(userSwitchingEventFilter, mUserLifecycleListener);
// 讲解注释⑧：只订阅"切换中"事件——用户切换
// 一经触发立刻重算 zone-user 表，见 :207
ExperimentalCarUserService experimentalUserService =
CarLocalServices.getService(ExperimentalCarUserService.class);
if (experimentalUserService != null) {
experimentalUserService.addPassengerCallback(mPassengerCallback);
} // 讲解注释⑨：副驾/后排免登录模式——乘客端
// 用户由系统代管，也要能收到创建/回收回调
mDisplayManager.registerDisplayListener(mDisplayListener,
new Handler(Looper.getMainLooper()));
// 讲解注释⑩：屏热插拔兜底（:224）——
// 车机的"插拔"多是上下电，不是物理拔线
ZoneUserBindingHelper helper = new ZoneUserBindingHelper() { // 讲解注释⑪
@Override
@NonNull
public List<OccupantZoneInfo> getOccupantZones(@OccupantTypeEnum int occupantType) {
... // 讲解注释⑫：按类型过滤返回区清单——给上游服务
} //  （如 CarAudioService）的"有哪些区"查询回调
@Override
public boolean assignUserToOccupantZone(@UserIdInt int userId, int zoneId) {
...
// 讲解注释⑬：落位三道闸（源码 :314-338）——同一用户
// 不得占两区、区必须存在、区已被其他用户占用则拒绝；
// 三闸全过才写 zoneConfig.userId 并返回 true
}
@Override
public boolean isPassengerDisplayAvailable() {
... // 讲解注释⑭：任一非 driver 区有可用主屏才返回
} // true——乘客娱乐功能的前置判定就问它
};
if (experimentalUserService != null) {
experimentalUserService.setZoneUserBindingHelper(helper); // 讲解注释⑮
} // ——绑定动作的"执行端"注册：⑬的三道闸经它生效
}
```

三处结构值得抠。**其一**，注释①不走 binder：`CarLocalServices.getService` 是进程内大杂烩（与书2 的 LocalServices 同型），同一进程内的服务互查不必绕 binder——第六章两阶段装配保证了此刻 CarPropertyService 已构造完成。**其二**，注释②是全章的锚点：驾驶座在哪，决定了 9.2 里 `seatSide=driver` 这个**相对语义**的绝对方向。**其三**，init 里三张表（区、屏、用户）一次建齐，之后靠三个监听（用户切换 :286、乘客回调 :292、显示器变化 :294）增量维护——静态解析只在启动做一次，动态归监听，与第三章电源状态机的"事件驱动"同构。

App 侧的查询接口以此表为底：`getDisplayForOccupant(zoneId, displayType)`（:507）按区+屏类型查 displayId，查不到返回 `Display.INVALID_DISPLAY`；`getDisplayIdForDriver(displayType)`（:543）则要求 `ACCESS_PRIVATE_DISPLAY_ID` 权限——**驾驶位的屏是私有屏**，普通 App 不给查。displayType 取值：DISPLAY_TYPE_MAIN=1（中控主屏）/DISPLAY_TYPE_CLUSTER=2（仪表）/DISPLAY_TYPE_INSTRUMENT_CLUSTER...（CarOccupantZoneManager.java:72 起）；occupantType：DRIVER=0/FRONT_PASSENGER=1/REAR_PASSENGER=2（:112-121）。

9.2 配置解析：OEM 用字符串表达整车形制

java packages/services/Car/service/src/com/android/car/CarOccupantZoneService.java:928-948（骨架+讲解注释）

```
// examples:
// <item>occupantZoneId=0,occupantType=DRIVER,seatRow=1,seatSide=driver</item>
// <item>occupantZoneId=1,occupantType=FRONT_PASSENGER,seatRow=1,
// searSide=oppositeDriver</item> // 讲解注释①：配置即文档——源码注释
// 给出的标准写法（第三行是源码原有的笔误）
boolean hasDriver = false;
int driverSeat = getDriverSeat(); // 讲解注释②：VHAL 报的驾驶座物理位形
int driverSeatSide = VehicleAreaSeat.SIDE_LEFT; // default LHD : Left Hand Drive
if (driverSeat == VehicleAreaSeat.SEAT_ROW_1_RIGHT) {
driverSeatSide = VehicleAreaSeat.SIDE_RIGHT;
} // 讲解注释③：左右舵判定——"driver"是相对概念，
// 右舵车（日本/英国）的 driver 在右，OEM 无需写死
int maxZoneId = OccupantZoneInfo.INVALID_ZONE_ID;
for (String config : res.getStringArray(R.array.config_occupant_zones)) {
int zoneId = OccupantZoneInfo.INVALID_ZONE_ID;
int type = CarOccupantZoneManager.OCCUPANT_TYPE_INVALID;
int seatRow = 0; // invalid row
String[] entries = config.split(","); // 讲解注释④：逐条解析 key=value 对
for (String entry : entries) {
String[] keyValuePair = entry.split("=");
switch (keyValuePair[0]) {
case "occupantZoneId": ... // 讲解注释⑤：四个合法键——
case "occupantType": ... // 区号/类型（DRIVER/FRONT_PASSENGER/
// REAR_PASSENGER 三选一）/座位排/
case "seatRow": ... // 座位侧（driver/oppositeDriver/
case "seatSide": ... // left/center/right），
// 非法键 throwFormatError 直接炸
}
}
```

> **设计洞察 · 设计洞察：为什么用资源字符串而非 Java 常量**
> OEM 适配的黄金法则：**改 overlay 不改代码**。五座轿车、七座 MPV、带双后排屏的行政版——形制差异全部收敛成 framework/base 里 config_occupant_zones 与 config_occupant_display_mapping 两个 string-array 的 overlay（RRO，CarService dump 会打印生效的 RRO 清单）。解析器对非法配置零容忍（throwFormatErrorInOccupantZones，:944 起）：缺 driver、双 driver、非法 key 都在 init 期直接抛——**配置错误死在启动而不是死在用户上车那一刻**，与 7.3 注释①"没有配置表宁可快速失败"是同一条工程信条。seatSide=driver 的相对语义（注释③）则把"左右舵"这个最容易翻车的产品差异消解在解析层。

多用户与区的绑定走双向：驾驶区固定跟随系统主用户（driver=当前前台用户），乘客区用户可由系统服务经 `setUserIdToOccupantZone` 指派（CarShellCommand.java:202），调试/产线可用 `cmd car_service set-user-id-to-occupant-zone <zoneId> <userId>` 与 `switch-user <id>`（:197）实时改写后观察 9.1 注释⑦的重算。

9.3 音区：CarAudioService 把声音挂到乘员区上

多屏解决了"谁看什么"，音区解决"谁听什么"：主驾导航播报不该从后排音响出声，副驾追剧不该污染主驾通话。CarAudioService（audio/CarAudioService.java:105）启动时经 CarAudioZonesHelper（:554，读 OEM 的 audio_control 配置 XML；旧版走 :573 的 Legacy 解析）构建 mCarAudioZones（:185），并把 **audioZoneId ↔ occupantZoneId** 映射（:556）与 **audioZoneId ↔ userId** 映射（:1373 的 setUserIdForAudioZoneLocked 及 :1465）挂进乘员区体系——用户切换时经 onOccupantZoneConfigChanged（:1540）联动重绑。上行链路复用第八章：每个音区独立走 AudioService 的焦点仲裁（同一音区内抢焦点，跨音区互不干扰），主音区为 PRIMARY_AUDIO_ZONE=0。

调试目标	命令（已核实，CarShellCommand）	作用
看全部分区	adb shell dumpsys car_service --services CarAudioService	音区表/音量组/用户映射全 dump
强制改 uid 的音区	adb shell cmd car_service set-audio-zone-for-uid \<uid\> \<zoneId\>	模拟"乘客 App 出错音区"复现
重绑区与用户	adb shell cmd car_service set-user-id-to-occupant-zone \<zoneId\> \<userId\>	验证 9.1 注释⑦的重算链
复现按键	adb shell cmd car_service inject-key \<keyCode\>	跨屏按键路由取证

> **设计洞察 · 交叉对照：三张"归属表"与第四章/书5 的分工**
> 书5 讲的是**窗口系统层**的多 DisplayContent（每屏一套焦点/栈），第四章讲了 CarActivityService 怎么用它们；本章是**语义层**：座位→屏→用户→音区四元组。WMS 只知道"displayId=2 上有个栈"，是 OccupantZone 表把 displayId=2 翻译成"副驾"。UI 策略层再叠一层：CarUxRestrictionsService 经装配表（6.3）依赖本服务，把驾驶态限制（第二章车速链）按**区**分发——同一时刻主驾屏禁视频、乘客屏放行，落点就是这里的区-屏映射。三层读通，AAOS 多屏就不再神秘。

回到全链：第六章的装配、第七八章的信号流、本章的归属表，共同构成附录 A 观测命令背后的完整世界。第九章习题（习题集 14/19 题）与排障手册场景三/四都从这里起跳。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **1. CarService 运行在什么进程、什么身份？给出至少三层证据。**

参考答案要点：独立 APK（com.android.car 进程，platform 签名），不是 system_server 的线程；CarServiceImpl extends ProxiedService（CarServiceImpl.java:40），以普通 Service 生命周期方式被拉起；binder 服务名 car_service 由 onCreate 里 addService 注册（:77）——App 经 Car.createCar 绑定。崩溃不拖 system_server（第一章）。

- **2. ICarImpl 为什么分"构造"与"init"两阶段？各阶段做什么？**

参考答案要点：构造期只建依赖图——new VehicleHal 与 30 余个子服务对象、完成引用注入；init 期统一"通电"——先 mHal.init() 建属性分发表，再按 mAllServices 顺序逐个 service.init()（ICarImpl.java:471-487）。一步到位会让顺序错误变成构造期崩溃；两阶段把顺序约束收敛到一张装配表（6.3）。

- **3. 把 0x11600207 按位分解，说出它是什么属性。**

参考答案要点：GROUP=0x1（SYSTEM，:130）+AREA=0x01（GLOBAL，:101）+TYPE=0x6（FLOAT，0x00600000）+INDEX=0x207，即 PERF_VEHICLE_SPEED = 0x0207 | SYSTEM | FLOAT | GLOBAL（types.hal:367-371）——车速，只读 CONTINUOUS。

- **4. VehiclePropertyChangeMode 三种取值各适合什么信号？与订阅的关系？**

参考答案要点：STATIC 永不变化（如 VIN），isPropertySubscribable（VehicleHal.java:660）直接拒绝订阅；ON_CHANGE 状态跳变型（门开关/档位），订阅即等推送；CONTINUOUS 连续量（车速），需采样率且被夹到 min/maxSampleRate 区间（types.hal:3856-3876）。

- **5. areaId 的语义是什么？全局属性与区域属性在配置上怎么区分？**

参考答案要点：areaId 是一个 AreaType（SEAT/WINDOW/DOOR…）下的区或区组合的位掩码，铁律是同一 AreaID 内的区永远同变、不同 AreaID 可独立控制（types.hal:77-86）；全局属性 areaConfigs 通常一项且 areaId 被忽略（:3997），区域属性每区一项、各带独立值域（VehicleAreaConfig:3995）。

- **6. App 注册 CarPropertyManager 监听后为什么往往"立即"收到一次回调?**

  参考答案要点：初始值补发——registerListener 末尾 mHandler.post(getAndDispatchPropertyInitValue)（CarPropertyService.java:303-306），全局属性补 1 帧、区域属性按 areaIds 逐区补（:309-321）；App 免手动 get，但要按 CarPropertyValue.timestamp 丢弃过期帧。

- **7. 三个 App 分别以 1/5/20 Hz 订阅车速，最终向 VHAL 下发的采样率是多少？20Hz 那个注销后呢?**

  参考答案要点：取最大 20Hz——registerListener 里仅当 rate 高于当前率才重订阅（CarPropertyService.java:288-290）；20Hz 注销后按剩余客户端的最大值 5Hz 重新订阅（unregister 对称路径）；实际生效值再被 config 的 min/maxSampleRate 夹逼。

- **8. 乘员区三要素是什么？写出一条 config_occupant_zones 配置项并解释各键。**

  参考答案要点：occupantZoneId（区号）/occupantType（DRIVER=0、FRONT_PASSENGER=1、REAR_PASSENGER=2）/座位位形（seatRow+seatSide）；示例 `occupantZoneId=0,occupantType=DRIVER,seatRow=1,seatSide=driver`（源码注释 CarOccupantZoneService.java:927）；seatSide=driver 是相对语义，随 VHAL INFO_DRIVER_SEAT 判定左右舵（:932-935）。

- **9. audio zone 与 occupant zone、user 是什么关系?**

  参考答案要点：CarAudioService 启动经 CarAudioZonesHelper（CarAudioService.java:554）建音区表，维护 audioZoneId 与 occupantZoneId 映射（:556）及与 userId 映射（setUserIdForAudioZoneLocked:1465）；用户切换经 onOccupantZoneConfigChanged（:1540）联动重绑；焦点仲裁按音区隔离——同音区内互抢焦点、跨音区互不干扰。

- **10. VHAL 进程死亡后 CarService 会发生什么？为什么这样设计?**

  参考答案要点：linkToDeath 注册的 VehicleDeathRecipient 回调 serviceDied/binderDied（CarServiceImpl.java:123/130）→ Process.killProcess 自杀 → init 重启 CarService 重走第六章启动链。强一致设计：VHAL 是车信号唯一事实来源，"半死"（部分属性失真）比全死更危险（第六章注释⑤）。

第二级：分析题（检验机制理解）

- **11. 完整追踪"车速事件从 vendor 到 UI 限制"的调用链，标注每次线程切换与 binder 跳点。**

  参考答案要点：CAN→vendor VHAL 进程→binder 回调→VehicleHal.onPropertyEvent（VehicleHal HandlerThread，:663）→PropertyHalService.onHalEvents（:408）→CarPropertyService.onPropertyChange（:622，锁外）→oneway binder→CarDrivingStateService（其 init 订阅 PERF_VEHICLE_SPEED/GEAR_SELECTION/PARKING_BRAKE_ON，CarDrivingStateService.java:71-73）→状态机计算→onDrivingStateChanged→CarUxRestrictionsManagerService（:731）→按 OccupantZone 区分发限制→SystemUI/App 消费。共两次跨进程（vendor→CarService、CarService→App），章节链：七→八→二→九。

- **12. VehicleHal.onPropertyEvent 与 CarPropertyService.onPropertyChange 都"锁内收集、锁外分发"，各自的死锁场景是什么？**

  参考答案要点：后者源码注释明说（:645-648）：onEvent 是 oneway binder，锁内调用时客户端若同步 unregister 会重入 mLock；前者锁外分发是因为 onHalEvents 进入各 HalServiceBase 后可能再取服务内部锁，锁序交叉风险。共同纪律：慢操作/可能重入的操作绝不持锁——聚批两跳是同一模式（8.2 ::box）。

- **13. OEM 配置了三个 occupant zone 但整机只有一块物理屏，getDisplayForOccupant 返回什么？下游哪些行为可预期？**

  参考答案要点：非匹配区返回 Display.INVALID_DISPLAY（CarOccupantZoneService.java:507-521）；下游乘客屏 App 启动失败或空白、音区-屏联动缺一块（handleActiveDisplaysLocked 只登记在位屏幕）、Cluster 类服务降级；系统不崩——DisplayListener（:224）在新屏接入时补齐映射（9.1 注释⑩）。

- **14. 对照书3 的 binder 十跳：App 调 createCar 到拿到 Manager 共发生哪些跨进程交互？断连怎么处理？**

  参考答案要点：轮询 ServiceManager 查 car_service（Car.java:1644）→AMS bindService→CarServiceImpl.onBind 返回 mICarImpl（:107-109）→onServiceConnected 里 asInterface 取代理并回调 onLifecycleChanged（:1312-1328）→getCarManager 再一次 IPC 建 Manager；断连时 resetCache+handleCarDisconnectLocked（:1786-1793）tearDown 全部 Manager——App 必须在 onLifecycleChanged(false) 时停用（8.3 表）。

- **15. CarDrivingStateService 为什么依赖 CarPropertyService 而不直接持有 VehicleHal？**

  参考答案要点：装配表证据（ICarImpl.java:419 注释）；分层理由——PropertyHalService 做管理属性↔VHAL 属性映射、权限闸与载荷校验（8.2 第二跳），绕过它直连 HAL 要重复实现这些且破坏"属性桥接单点"（7.3 PropertyHal 最后 init 的原因）；代价一层转发，收益校验收口一处。

- **16. 若装配表把 CarPropertyService 排到 CarUxRestrictionsManagerService 之后，会在哪一步、以什么形式失败？**

  参考答案要点：构造期未必崩——构造顺序由构造器实参决定（:249），与 add 顺序解耦；真正失败在 init 期：UxR init 时订阅依赖的属性链未就绪（mHal.init 已先跑但服务 init 顺序错乱），典型症状是"驾驶态永不更新"的静默故障。考点：构造顺序与 init 顺序是两条线，装配表管的是后者（6.3 两阶段动机）。

- **17. SHUTDOWN_PREPARE 阶段用户重新点火（取消关机）会发生什么？状态机如何保证不撕裂？**

  参考答案要点：走 SHUTDOWN_CANCELLED 回到 ON（PowerHalService.java:78，第三章状态表）；CarPowerManagementService 是"vendor 上报 AP_POWER_STATE_REQ（0x11510A00）→框架以 AP_POWER_STATE_REPORT（0x11510A01）应答"的单线程事件驱动状态机，取消也是事件流中的一条，天然无并发撕裂；挂起中的 GarageMode 等经 cancelled 通知回收。

第三级：设计题

- **18. 设计"订阅风暴防护"：N 个 App 高频注册/注销同一 CONTINUOUS 属性时的资源与抖动问题，给出至少三个机制点并标注源码落点。**

参考答案要点：①频率天花板——min/maxSampleRate 夹逼（types.hal:4040-4047）+框架侧 getSampleRate 记账；②注册路径出锁——mHandler.post（CarPropertyService.java:303）已隔离慢路径，可再加注销延迟合并（最后一名注销后 N ms 才真正降频 unsubscribe）；③初始值补发去重——mClientMap 按 binder 键去重（:273-279）保证同 client 不重复补帧；④观测面——dumpsys car_service --services CarPropertyService 暴露 client/propId/率 表，回归测试用 inject-continuous-events 打流量（CarShellCommand.java:165）。

- **19. 为"七座 MPV+三屏（仪表/中控/后排吸顶）+右舵"写出 occupant zones 配置草案，并说明右舵为何无需单独配置。**

参考答案要点：`occupantZoneId=0,occupantType=DRIVER,seatRow=1,seatSide=driver`、`occupantZoneId=1,occupantType=FRONT_PASSENGER,seatRow=1,seatSide=oppositeDriver`、`occupantZoneId=2,occupantType=REAR_PASSENGER,seatRow=2,seatSide=left`；display mapping 把 driver 区配 cluster+main、rear 区配吸顶屏 MAIN。右舵自动生效：seatSide=driver/oppositeDriver 是相对语义，解析期由 INFO_DRIVER_SEAT=SEAT_ROW_1_RIGHT 翻译成 SIDE_RIGHT（:932-935），overlay 一字不改。

- **20. 假设把 CarService 合并进 system_server，列出至少三个必须重新解决的问题。**

参考答案要点：①崩溃域——Car*Service 崩溃即重启整个 system_server，第一章崩溃隔离收益归零；②VHAL 死亡策略——killProcess 自杀（第六章注释⑤）在 system_server 里等价整机重启，必须改局部重连；③升级节奏——CarService 是可独立更新的 APK（dump 可见其包名），合并后与镜像绑死；④线程模型——init 的 @MainThread 假设撞上 system_server 主线程预算与 watchdog；⑤SELinux——独立 car 域并入 system_server 域，vendor 回调路径的访问控制要重新设计。

术语表

（按术语拼音排序；源码锚点以 Android 13 为准。）

术语	解释
areaId	车信号的作用区编号：一个 AreaType（SEAT/WINDOW/DOOR…）下区或区组合的位掩码；同一 AreaID 内的区必须永远同变，不同 AreaID 必须可独立控制（types.hal:77-86）
CarServiceBase	CarService 全部子服务的接口三件套：init/release/dump；与书13 CoreStartable 同构（第一章）
CarUxRestrictions	驾驶分心限制：由 CarUxRestrictionsManagerService 依据 DrivingState（车速/档位/手刹）按乘员区分发给 App 的 UI 约束

术语	解释
HalServiceBase	VHAL 框架侧"属性认领者"基类，9 个实现（Power/Input/Property/User/Vms/Diagnostic/Cluster/Evs/Time），VehicleHal.java:166-176
OccupantZone	乘员区：座位位形+occupantType 定义的"一个人"抽象，向下绑 displayId/userId/audioZoneId（第九章）
occupantType	乘员类型：DRIVER=0/FRONT_PASSENGER=1/REAR_PASSENGER=2（CarOccupantZoneManager.java:112-121）
onPropertyEvent	VHAL 上行事件回调：vendor→VehicleHal（:663）→HalServiceBase→App 的推送链入口（第八章）
ProxiedService	CarServiceImpl 的基类（:40）：以普通 Service 身份被拉起、把 binder 转发到 ICarImpl 的"代理服务"
VehicleArea	属性 ID 的区域类型位段（掩码 0x0f000000）：GLOBAL/WINDOW/SEAT/DOOR 等（types.hal:99-114）
VehicleAreaConfig	区域级配置：areaId+该区合法值域（min/max），同一属性可按区设不同范围（types.hal:3995）
VehiclePropConfig	属性元数据表：prop/access/changeMode/areaConfigs/采样率界——属性的静态能力说明（types.hal:4019）
VehiclePropValue	属性值四元组：timestamp+areaId+prop+status+RawValue，车信号在进程间的唯一载体（types.hal:4065）
VehiclePropertyChangeMode	变化模式：STATIC（不可订阅）/ON_CHANGE（跳变推送）/CONTINUOUS（按采样率推送）（types.hal:3856）
VehiclePropertyStatus	属性健康度：AVAILABLE/UNAVAILABLE/ERROR；是值的字段而非配置的字段（types.hal:3899）
VHAL	Vehicle HAL：把 CAN/车身信号抽象为属性的 vendor 进程；框架侧客户端是 VehicleHal.java:71
采样率协商	CONTINUOUS 属性多客户端订阅时取最大请求率、被 config 的 min/maxSampleRate 夹逼（CarPropertyService.java:288-290）
初始值补发	注册监听后立刻推送当前值：全局属性 1 帧、区域属性逐区补（CarPropertyService.java:309-321）
驾驶状态	DrivingState：PERF_VEHICLE_SPEED+GEAR_SELECTION+PARKING_BRAKE_ON 三属性合成的 fully-idle/idling/moving 状态机（CarDrivingStateService.java:71-73）
属性分发表	mPropertyHandlers：propId→HalServiceBase 的路由表，init 期构建、事件回调期反查（VehicleHal.java:235/245，:670）
死亡陪葬	VHAL 进程死亡→VehicleDeathRecipient 让 CarService 自杀重启的强一致策略（CarServiceImpl.java:120-135）

术语	解释
音区	audio zone：CarAudioService 的声音分区，audioZoneId 与乘员区/用户双向绑定，焦点按区隔离（第九章 9.3）
装配表	mAllServices：ICarImpl 构造期按依赖顺序声明的子服务数组，init 正序/release 逆序（ICarImpl.java:457，：471-497）

排障手册：车机 AAOS问题定位

1. dumpsys 定位状态 → 2. logcat 查看日志 → 3. 深入各章。

深度实战：车机 AAOS的排障场景分析

场景一：车机屏幕不亮（电源状态机卡在 WAIT_FOR_VHAL）

症状：点火后中控/仪表黑屏或停在开机动画；boot.car_service_created 已置 1 但屏幕不亮，或 CarService 日志反复出现等待电源状态的输出。

定位命令：

text 电源状态机卡点定位（命令均已对照源码核实）

```
adb shell dumpsys car_service --services CarPowerManagementService
adb logcat -s CAR.POWER -s CAR.SERVICE
adb shell cmd car_service get-property-value 0x11510A00
adb shell dumpsys SurfaceFlinger | grep -i display
```

根因：第三章状态机的第一道闸是 WAIT_FOR_VHAL——vendor 必须上报 AP_POWER_STATE_REQ（0x11510A00，types.hal:1517，INT32_VEC 载荷：int32Values[0]=状态枚举、[1]=参数）。卡死的三种典型：VHAL 根本没注册（get-property-value 直接报错，对照场景五）；上报的 state/param 越界（如未知枚举，CarPowerManagementService 拒绝迁移）；SHUTDOWN_PREPARE 阶段某子服务 release/回调阻塞，状态机停在中间态。SurfaceFlinger 分节用于区分"框架没让亮"与"WMS/显示层没亮"。

修复：开发期用 default VHAL 或 VHAL 模拟器正确发送状态序列（WAIT_FOR_VHAL→ON）；产线核对 vendor 实现对 AP_POWER_STATE_REPORT（0x11510A01）的应答是否及时——REPORT 晚了框架会按超时策略处理；`cmd car_service suspend/resume/hibernate`（CarShellCommand.java:175-177）可在设备上手动驱动状态机复现。

场景二：App 收不到属性事件（订阅后无回调）

症状：registerCallback 返回成功，但车速/档位等回调从不触发；同一属性系统应用正常。

定位命令：

text 属性事件链五级排查（链路见第八章 8.1/8.2）

```
adb shell dumpsys car_service --services CarPropertyService
adb shell dumpsys car_service --hal PropertyHalService
adb shell cmd car_service get-carpropertyconfig
adb shell cmd car_service inject-vhal-event 0x11600207 30
adb logcat -s CAR.HAL -s CAR.SERVICE
```

根因：逐闸排查。其一，权限——registerListener 的 assertPermission（CarPropertyService.java:266）缺 READ 权限直接抛 SecurityException，App 端常吞掉异常显得"静默"；其二，属性不可订阅——STATIC 或不可读被 isPropertySubscribable 拒绝（VehicleHal.java:660），get-carpropertyconfig 可查 changeMode；其三，App 侧 Manager 失效——CarService 曾重启，断连把全部 Manager tearDown（Car.java:1786），App 没监听 onLifecycleChanged 就永远收不到；其四，vendor 不上报———-hal 分节里该属性的事件计数/最后时戳不动（7.3 注释④的 mEventLog 记账）；其五，事件到了但无认领者——logcat 出现 "HalService not found for prop"（VehicleHal.java:675）。

修复：inject-vhal-event（CarShellCommand.java:163，语法 **<PROPERTY_ID 十六进制或十进制> [zone] data [-t 延时秒]**）做二分：注入后有回调=订阅链健康、问题在 vendor 上报侧；无回调=逐查权限/changeMode/Manager 存活。vendor 侧修 onSet/onChange 上报；App 侧按 8.3 的生命周期范式重写注册时机（onResume 重建）。

场景三：音区路由错误（副驾的声音从主驾音箱出）

症状：乘客屏 App 播放音乐，声音却从中控扬声器出；或主驾导航播报整舱响。

定位命令：

text 音区绑定排查（命令均已核实）

```
adb shell dumpsys car_service --services CarAudioService
adb shell cmd car_service set-audio-zone-for-uid <uid> <zoneId>
adb shell cmd car_service set-user-id-to-occupant-zone <zoneId> <userId>
adb logcat -s CAR.AUDIO
```

根因：CarAudioService 的三级映射有一级断了——uid→audioZone（按 uid 路由的兜底表，CarShellCommand.java:184 的 set-audio-zone-for-uid 即改它）、userId→audioZone（setUserIdForAudioZoneLocked，CarAudioService.java:1465）、audioZone↔occupantZone（:556，随乘员区配置装载）。用户切换后 onOccupantZoneConfigChanged（:1540）没触发或晚触发，重绑缺位是最常见根因；其次是 OEM 的 audio 配置 XML 里 bus 与 zone 归属写错（CarAudioZonesHelper，:554）。

修复：dumpsys 的 AudioZoneId 与 OccupantZoneId/User 映射分节（:356-368 的 dump 内容）逐项核对；用 set-audio-zone-for-uid 手工重绑验证路由恢复；错位若只在切换后出现，追 CarOccupantZoneService 的 SWITCHING 监听链（9.1 注释⑦/⑧）与用户切换时序。

场景四：多用户切换后乘员区显示错乱

症状：switch-user 后乘客屏仍显示主驾账号的应用；或 driver 区的 userId 查询返回旧值，Launcher/收藏夹"串号"。

定位命令：

text 乘员区用户绑定排查（命令均已核实）

```
adb shell dumpsys car_service --services CarOccupantZoneService
adb shell cmd car_service switch-user <userId>
adb shell cmd car_service set-user-id-to-occupant-zone <zoneId> <userId>
adb logcat -s CAR.SERVICE -s CAR.USER
```

根因：CarOccupantZoneService 只订阅 USER_LIFECYCLE_EVENT_TYPE_SWITCHING（init：286-291）——监听漏了、或 CarUserService 切换流程中断，zone-user 重算（handleUserChangesLocked，:283）就不会发生；getDriverUserId 基于的重算结果过期；ExperimentalCarUserService 缺失时乘客回调（:292）整组失效。另注意 guest/profile 生命周期策略与 zone 绑定的冲突（profile 用户被回收时区表必须清空）。

修复：等 SWITCHING 完成态再断言 dump；手工 set-user-id-to-occupant-zone 纠正并确认 dumpsys 分节更新，即可把问题二分为"重算没跑"或"重算跑错"；重算没跑就追 CarUserService 的事件分发，跑错就核对该 OEM 的用户策略（9.2 配置层）。

场景五：CarService 反复重启（VHAL 死亡循环）

症状：com.android.car 进程周期性消失重启，logcat 反复出现 "***Vehicle HAL died. Car service will restart***"（CarServiceImpl.java:125）；所有车信号类 App 表现为间歇性失联。

定位命令：

text VHAL 死亡循环取证

```
adb logcat -s CAR.SERVICE:W AndroidRuntime:E
adb shell dumpsys car_service --version
adb shell dumpsys car_service --list-hals
ps -A | grep -i vehicle
```

根因：VehicleDeathRecipient 的 serviceDied/binderDied（:123/:130）执行 Process.killProcess 自杀（第六章注释⑤的强一致设计）——它救不了"VHAL 本身反复崩"的设备：CarService 重启→重连→vendor 再死→再自杀，循环。另一类根因是 CarService 自身崩溃（AndroidRuntime 缓冲区分堆栈首帧），如 OEM 缺配置导致 init 抛 RuntimeException（7.3 注释①、9.2 的 throwFormatError）。

修复：先用 --version 对齐框架与 vendor 的 AIDL/HIDL 接口版本（接口错配是 linkToDeath 立死的常见原因）；vendor 侧修稳定后循环自解；临时缓解可切 default VHAL 实现验证框架侧无罪；CarService 自身崩溃按堆栈对照第六章启动链逐服务排查。

车机 AAOS 核心速查表

机制	一句话	源码锚点	所在册
CarService	独立 APK（com.android.car），ProxiedService:40	packages/services/Car	本册

机制	一句话	源码锚点	所在册
VHAL	VehicleHal:71 属性订阅/分发（车信号统一语言）	car/hal/VehicleHal.java	本册
电源差异	熄火≠关机（CAN 唤醒，强制恢复）	CarPowerManagementService:103	本册
多屏	CarActivityService 管多 DisplayContent	CarActivityService	本册
DrivingState	车速→禁止视频（VHAL→状态→Policy→UI）	CarDrivingStateService	本册